SENIOR DATA / BACKEND ENGINEER · PORTFOLIO

7 / 7 requirements covered · Live demo + source code public

MarketIntel Implementation Manual

A working market-intelligence platform built end-to-end against a Senior Data / Backend Engineer role specification — from third-party ingestion to a partitioned dimensional warehouse to an interactive dashboard.

AUTHOR

Humberto Henriquez

REPO

github.com/henriquezbh5-cpu/marketintel

VERSION

v1.0 · 2026-04-25

Executive summary

MarketIntel is a production-shaped demo built specifically against the seven role requirements. It is not a toy: real ingestion runs every minute against Yahoo Finance, data lands in partitioned PostgreSQL, the dashboard polls live, and 27 tests gate every change.

How the role spec maps

#	REQUIREMENT	STATUS	WHERE IT LIVES
1	Scalable Python/Django backend	DELIVERED	apps/api, apps/dashboard, apps/core
2	Third-party ingestion pipelines	DELIVERED	pipelines/connectors, pipelines/tasks
3	Data modelling and schema design	DELIVERED	apps/instruments, apps/prices, apps/news
4	Async / distributed processing	DELIVERED	config/celery.py, pipelines/tasks/common.py
5	Infra orchestration / storage / warehousing	DELIVERED	pipelines/orchestration, warehouse, infra/k8s
6	Cross-functional collaboration	SOLO DEMO · STRUCTURED FOR TEAM-FIT	docs/CONTRIBUTING, docs/RUNBOOK, .github/
7	Strategic decisions with the CTO	CAPTURED IN 7 ADRS	docs/adr/0001..0007.md

Headline numbers

Code surface	~150 source files across apps, pipelines, infra and tests
Tests	27 / 27 passing · 74% line coverage on apps + pipelines
Architecture decisions	7 ADRs documenting trade-offs (Django, Celery, Dagster, partitioning, DuckDB, etc.)
Live data	18 instruments · 21,000+ OHLCV bars across 5 resolutions
Refresh cadence	Spot quotes every 60s · Gold matviews every 5 min
Containers	7 (postgres · redis · minio · web · worker · beat · dagster)

Table of contents

01 Architecture at a glance page 4

02 Requirement 1 — Scalable Python/Django backend page 6

03 Requirement 2 — Third-party ingestion pipelines page 8

04 Requirement 3 — Data modelling and schema design page 10

05 Requirement 4 — Async / distributed processing page 12

06 Requirement 5 — Infrastructure: orchestration, storage, warehousing page 14

07 Requirement 6 — Cross-functional collaboration page 16

08 Requirement 7 — Architecture strategy with the CTO page 17

09 Operations manual — running, testing, deploying page 18

10 Live demo URLs and what each page shows page 21

11 Source-code map page 22

1 · Architecture at a glance

A single sentence: third-party API responses flow through Bronze (immutable raw on MinIO) into Silver (normalised, partitioned PostgreSQL), and Gold (refreshed materialised views and DuckDB analytics) — orchestrated by Dagster and powered by a Celery worker pool, served by Django through both a REST API and an interactive dashboard.

The seven physical containers

```
┌──────────┐   publishes   ┌──────────┐
│  beat    │ ──────────────>│  redis   │  broker · cache · result backend
│ scheduler│                └──────────┘
└──────────┘                     │ pulls
                                 v
                          ┌──────────┐   HTTP   ┌──────────────────┐
                          │ worker   │ ────────> │ Yahoo Finance    │
                          │ Celery   │ <──────── │ (third party)    │
                          └──────────┘           └──────────────────┘
                                 │ UPSERT
                                 v
┌──────────┐                ┌──────────┐                  ┌──────────┐
│ minio    │ <── bronze ──┤ postgres │ ── ORM SELECT ->│  web     │
│ bronze   │   JSONL.gz     │ silver   │                  │ Django   │
└──────────┘                └──────────┘                  │ + DRF    │
                                 │                        │+dashboard│
                                 │                        └──────────┘
                            refresh│ matviews                 │ HTTP
                                 v                            v
                          ┌──────────┐                  ┌──────────┐
                          │ gold     │                  │ Browser  │
                          │ matviews │                  │ (recruiter │
                          │ + DuckDB │                  │ / CTO)   │
                          └──────────┘                  └──────────┘

┌──────────┐
│ dagster  │ ── orchestrates the asset graph (separate UI)
└──────────┘
```

The Medallion data layout

LAYER	STORAGE	MUTABILITY	PURPOSE
Bronze	MinIO/S3 — JSONL.gz	Append-only, immutable	Replay deterministic. Audit. Bug-debug transformations
Silver	PostgreSQL 16 — partitioned	Idempotent UPSERT	Serves the API and the dashboard. Joins, indexes, FKs
Gold	Postgres matviews + DuckDB	Refreshed on schedule	Hot dashboard queries · heavy analytics isolated from OLTP

What runs every minute

1. **Beat** publishes a message to Redis on the `ingest` queue.

2. An idle **Celery worker** picks the message up and instantiates the Yahoo Finance connector.
3. The **connector** applies a token-bucket rate limit, checks the circuit breaker, then issues an HTTP GET via `httpx`.
4. Yahoo responds with JSON; the connector parses it into a typed dataclass.
5. The worker persists the data with `INSERT ... ON CONFLICT DO UPDATE` on the natural key, making the operation idempotent.
6. Within 5 seconds, the dashboard's polling cycle fetches the new data via a Django view that reads ORM directly — no internal HTTP roundtrip.
7. The chart and sparklines re-render. The user sees the price update.

End-to-end latency from upstream change to user screen: **~10 seconds at worst**.

1

BACKEND · APPLICATION LAYER

Scalable Python/Django backend

DELIVERED

> *"Design, build, optimise and maintain scalable features and systems in our Python/Django backend."*

What was built

A Django 5 application split into six apps (`core` , `instruments` , `prices` , `news` , `api` , `dashboard`) plus the `pipelines` Django app that owns Celery tasks. Settings are environment-split (`base · dev · prod · test`) so the same image runs locally, in CI and in production with only environment variables changing.

Key design choices

- **API key authentication** with hashed-at-rest keys (`sha256`). Keys are minted via `scripts/issue_api_key.py` , shown to the user once, and never persisted in plain text. Scopes are `read` / `write` / `admin` .
- **Per-key throttling**: the custom `APIKeyScopedThrottle` derives the cache bucket from the API key's primary key, not the user's, so a single user with multiple keys gets independent rate budgets — useful when the same user shares one key across services.
- **Pagination split by use case**: cursor pagination (keyset on `ts`) for time-series endpoints — O(log N) regardless of depth — and limit/offset for catalogues (small enumerable lists).
- **Health probes** shaped for Kubernetes: `/health/live/` never touches dependencies (just confirms the process is up), while `/health/ready/` validates Postgres + Redis. The kubelet uses these to manage rollouts and restarts.
- **Hardening flags**: `EXPOSE_ADMIN` and `EXPOSE_METRICS` environment variables gate the `/admin` and `/metrics` routes so the same image can be exposed publicly via tunnel or deployed to staging without leaking sensitive endpoints.
- **Structured logging**: a custom `JSONFormatter` emits one JSON line per record. The `RequestIDMiddleware` threads `X-Request-ID` from the load balancer through the whole stack via `ContextVar` , so every log line in a request — Django, ORM, Celery if dispatched — shares the same correlation id.

Where you see it

- Live API documentation: `/api/docs/` renders the OpenAPI 3.1 schema as Swagger UI
- Health probes: `/health/live/` and `/health/ready/` return JSON status

- The dashboard at `/` is itself a Django app — server-rendered templates with Alpine.js + Chart.js for live updates

```
apps/api/ · apps/core/health.py · apps/core/middleware.py · apps/core/logging.py ·
config/settings/{base,dev,prod,test}.py
```

2

INGESTION · THIRD-PARTY DATA

Third-party ingestion pipelines

DELIVERED

> *"Design and build robust data ingestion pipelines to incorporate, normalise and continuously refresh the third-party data sources that power our platform."*

What was built

A connector framework with a shared base (`BaseConnector`) and four concrete adapters. The currently active source is **Yahoo Finance** for equity quotes and OHLCV; three additional adapters (Binance, CoinGecko, CryptoPanic) are committed and ready to activate.

The base layer (where the robustness lives)

- **Token-bucket rate limiting** per connector instance. Configurable via env (`YAHOO_RATE_LIMIT_PER_MIN`, `BINANCE_RATE_LIMIT_PER_MIN`, etc.). When the bucket is empty, the connector raises `RateLimited` with a `retry_after` hint so the task can re-enqueue with backoff rather than busy-waiting.
- **Circuit breaker** (closed / open / half-open). After 5 consecutive failures the breaker opens for 30 seconds; calls fail fast with `CircuitOpen`. After the cooldown a single probe is allowed; success closes the breaker, another failure opens it again. Prevents storming a degraded upstream.
- **Typed errors**: `RateLimited`, `CircuitOpen`, `ConnectorError`. Tasks declare these in `autoretry_for` so transient failures retry automatically while structural errors fail immediately.
- **Prometheus metrics** per request (counter by `source × status`, latency histogram, breaker-open counter) wired through the base — adapters get observability for free.

What the Yahoo adapter does specifically

- Calls the public chart endpoint `query1.finance.yahoo.com/v8/finance/chart/<symbol>` — no API key, no SDK dependency.
- Sends a browser-flavoured `User-Agent` so the anonymous JSON keeps flowing.
- Parses the `meta` block into a `YFQuote` dataclass and the timestamp/quote arrays into `YFCandle` records.
- Skips bars where any OHLCV value is null (Yahoo emits null for trading-halt minutes — silver should not see those).

How "continuously refreshed" is guaranteed

- **Beat schedule** fires `ingest_yahoo_spot_for_active` every minute on the `ingest` queue.
- **Idempotent UPSERTs**: `INSERT INTO fact_price_spot ... ON CONFLICT (instrument_id) DO UPDATE`. Re-running over the same window updates in place — no duplicates, no races.
- **Live evidence**: the dashboard's `/system/` page reports last-seen timestamp per source and labels it `LIVE` if the last event was within 10 minutes, `STALE` otherwise.

```
pipelines/connectors/{base,yahoo_finance,binance,coingecko,cryptopanic}.py ·
pipelines/tasks/yahoo.py
```

3

SCHEMA · DATA MODELLING

Data modelling and schema design

DELIVERED

> *"Own data modelling and schema design across our platform, making sure the data layer is structured to deliver performance, flexibility and long-term scalability."*

The dimensional model

Star schema with two dimensions and three facts:

```
    dim_source (small, mostly static)
        |
        v
    dim_instrument (SCD Type 2)        — Apple Inc., NVIDIA, S&P 500, etc.
        |
   ┌────┼──────────────────┐
   v    v                  v
   fact_price_spot   fact_price_candle  fact_news
   (one row per      (partitioned by    (M2M back to
    instrument)       month, billions)   instruments)
```

SCD Type 2 on the instrument dimension

Tickers change. Companies rebrand. Securities delist. The `Instrument` table tracks every version with `valid_from`, `valid_to` and `is_current`. The active row is enforced via a **partial unique index**:

```
UniqueConstraint(
    fields=["source", "external_id"],
    condition=Q(is_current=True),
    name="uniq_current_instrument_per_source",
)
```

This is a Postgres-native feature that Django supports first-class — no triggers, no manual enforcement. The service layer (`upsert_instrument_scd2`) compares tracked attributes (symbol, name, asset_class, quote_currency) and either no-ops, updates metadata in place, or closes the previous row and opens a new current version.

Time-series fact partitioning

`fact_price_candle` is a Postgres `PARTITION BY RANGE (ts)` table with monthly partitions. Currently 10 active partitions covering Oct 2025 → Jul 2026; future partitions are auto-created by a Celery maintenance task on the 25th of every month.

The PK includes the partition key, as Postgres requires:

```
PRIMARY KEY (instrument_id, source_id, resolution, ts)
```

Index strategy per partition:

- **BRIN on `ts`** — Block Range Index, ~100× cheaper than B-tree for append-only time-series, ideal because rows arrive in monotonic `ts` order.
- **Composite B-tree on `(instrument_id, ts DESC)`** — the dominant query pattern is "give me the last N candles for symbol X". This index makes that O(log N) per partition with an index-only scan possible.
- **Resolution + ts** — for resolution-scoped dashboards.

Why declarative partitioning over Citus / TimescaleDB

Documented in **ADR 0005**. Stock Postgres meets the requirements at our scale (multi-billion rows projected); adding TimescaleDB adds extension dependency and license footnotes; Citus solves a distributed problem we don't have. The partitioning + BRIN combination is good for ~10 TB on a single node, plenty of runway.

Gold materialised views

Three matviews refreshed concurrently every 5 minutes by Celery. Each has a unique index (required for `REFRESH MATERIALIZED VIEW CONCURRENTLY`):

- `gold.mv_top_movers_24h` — top gainers / losers, sorted by change %
- `gold.mv_volume_leaders_24h` — highest 24h notional volume
- `gold.mv_news_sentiment_daily` — aggregate sentiment per symbol per day

These power dashboard queries that would otherwise scan the partitioned fact every refresh.

```
apps/instruments/models.py · apps/prices/models.py · apps/news/models.py ·
apps/prices/migrations/0002_partition_candles.py · infra/postgres/post_migrate/01_gold_marts.sql
· docs/data-model.md · docs/adr/0005-postgres-partitioning.md
```

4

ASYNC · DISTRIBUTED

Async / distributed processing

DELIVERED

> *"Implement async and distributed processing to manage large-scale operations reliably and efficiently."*

Celery + Redis topology

Celery 5 with Redis as broker and result backend. Four named queues route work by domain:

QUEUE	CARRIES	WORKER BEHAVIOUR
`default`	Maintenance, generic work	Standard prefork pool
`ingest`	All third-party API calls	Throttled by connector token buckets
`transform`	Gold matview refreshes, normalisation	Single-flight per matview
`dlq`	Manually requeued failures	Operator-driven, not Beat-driven

Reliability guarantees on every task

The `IdempotentTask` base class (`pipelines/tasks/common.py`) sets sensible defaults that all pipeline tasks inherit:

- **Acks late**: a task is only acked after it succeeds. If a worker dies mid-task, Redis re-delivers the message to another worker.
- **Reject on worker lost**: same idea, surfaces sigkill scenarios.
- **Prefetch = 1**: workers don't hoard; tasks are distributed evenly across the pool.
- **Retries with exponential backoff and jitter**: `retry_backoff=True` , `retry_jitter=True` , `max_retries=5` , capped at 10 minutes.
- **Auto-retry on transient errors**: `HTTPError` , `ConnectorError` , `RateLimited` . 4xx errors don't retry.
- **Soft + hard time limits**: `soft_time_limit` raises `SoftTimeLimitExceeded` in-task so cleanup runs; `time_limit` hard-kills at 30s past that.

Dead-letter queue (the bit most demos skip)

When a task exhausts its retries, the `on_failure` hook persists everything to the `core_dead_letter_task` Postgres table:

```
DeadLetterTask.objects.update_or_create(
    task_id=task_id,
    defaults={
        "task_name": self.name,
        "queue": (self.request.delivery_info or {}).get("routing_key", ""),
        "args": list(args or []),
        "kwargs": dict(kwargs or {}),
        "exception": repr(exc)[:8192],
        "traceback": str(einfo)[:16384] if einfo else "",
        "attempts": self.request.retries + 1,
    },
)
```

The Django admin has a custom action — "*Requeue tasks (idempotent)*" — that re-publishes the failed messages to their original queue. Because every task is idempotent, requeue is always safe.

Beat schedules driving the platform

CADENCE	TASK	QUEUE
Every minute	`ingest_yahoo_spot_for_active`	ingest
Every 5 minutes	`refresh_gold_marts`	transform
Every 5 minutes	`ingest_news_recent`	ingest
25th of month	`ensure_future_partitions`	default

`config/celery.py · pipelines/tasks/common.py · pipelines/tasks/yahoo.py · apps/core/models.py (DeadLetterTask) · apps/core/admin.py`

5

INFRASTRUCTURE · ORCHESTRATION · STORAGE

Infrastructure: orchestration, storage, warehousing

DELIVERED

> *"Own the data infrastructure tooling, including pipeline orchestration, storage and warehousing, together with development of the application layer."*

Why Dagster + Celery (not just one)

These tools serve different purposes and complement each other.

	DAGSTER	CELERY
Mental model	Asset graph, declarative	Task queue, imperative
Granularity	Pipeline / dataset	Function / unit of work
Schedules & sensors	First-class	Beat (limited)
Lineage / metadata	First-class	None
Massive parallelism	Limited	Unlimited (horizontal workers)

Dagster owns the *asset DAG*: `bronze_yahoo_spot` → `silver_price_spot`. Heavy fan-out delegates to Celery via `group()`. Dagster waits on the result and publishes the asset materialisation. ADR 0003 documents the trade-off.

Storage tiers

TIER	TECH	WHAT LIVES THERE
Object storage (bronze)	MinIO (S3-compatible) — swappable for AWS S3, Backblaze B2, R2	Raw third-party JSON, gzip-compressed JSONL, partitioned `source/entity/dt/hh`
OLTP / silver	PostgreSQL 16	Normalised facts and dimensions; the hot path the API serves from
Analytics	DuckDB embedded in worker processes	Heavy queries (correlation matrices, factor exposures) read Postgres via the `postgres_scanner` extension; isolated from OLTP
Cache / broker	Redis	Celery broker + result backend, throttle counters, short-TTL spot cache

Application layer

Two surfaces backed by the same models:

- **REST API** at `/api/v1/` for programmatic consumers — full OpenAPI 3.1 schema, scoped throttle, cursor pagination for time-series.
- **Interactive dashboard** at `/` for humans — server-rendered Django templates with Tailwind + Alpine + Chart.js, polling JSON endpoints every 5–15 seconds. The dashboard reads ORM directly (no roundtrip to the API), keeping latency low.

Production deployment

Three deploy paths committed in the repo:

- `render.yaml` blueprint — free tier, auto-provisions Postgres + Redis + web + worker.
- `railway.toml` — Railway platform, with notes for adding plugins and worker service.
- `infra/k8s/` — Kubernetes manifests with Kustomize base + staging/production overlays. Includes Deployment, Service, HPA (CPU + memory), PodDisruptionBudget, NetworkPolicy (deny-all-default + targeted allows), Ingress with cert-manager, and a one-shot migrations Job.

CI/CD

- GitHub Actions: lint (ruff, mypy advisory) + test (pytest with services for Postgres + Redis) + build (multi-stage Docker, push to GHCR).
- CodeQL weekly security scans (Python).
- Dependabot grouped updates (django-stack, celery-stack, dagster-stack, observability).

Observability

- Prometheus metrics: connector latency by source, queue depth, DLQ inserts, freshness gauges.
- Two pre-built Grafana dashboards (`marketintel-overview.json`, `marketintel-pipelines.json`).
- Prometheus alerts (`infra/prometheus/alerts.yml`): high error rate, p95 latency breach, freshness stale, DLQ growth.
- Optional OpenTelemetry tracing wired in `config/observability.py` (Django + Celery + httpx + psycopg auto-instrumented when `OTEL_EXPORTER_OTLP_ENDPOINT` is set).

```
pipelines/orchestration/{assets,jobs,sensors}.py · warehouse/db.py · warehouse/queries/*.sql ·
infra/k8s/ · render.yaml · railway.toml · .github/workflows/ · infra/grafana/ ·
infra/prometheus/
```

6

CROSS-FUNCTIONAL COLLABORATION

Working with developers, AI engineers, data scientists

SOLO DEMO · STRUCTURED FOR TEAM-FIT

> *"Contribute to new product features and integrations, working closely with other developers, AI engineers and data scientists."*

Honest framing

This is a one-person demo, so collaboration cannot be *shown*. What can be shown is that the project is **structured to make collaboration smooth from day one**.

For other developers joining the codebase

- `docs/CONTRIBUTING.md` — setup in three commands (`make install-dev`, `make up`, `make migrate`), branching convention, testing matrix, walk-through for "adding a new connector".
- `.github/PULL_REQUEST_TEMPLATE.md` — every PR must answer *what · why · how · test plan · risk · rollback*. Filters drive-by changes.
- `.github/CODEOWNERS` — schema-touching changes (`migrations/`, `infra/postgres/`, `docs/data-model.md`) require explicit review.
- **27 tests + CI gates**: PRs cannot merge red.
- **Pre-commit hooks** (ruff, ruff-format, gitleaks, trailing-whitespace) run on every commit so issues surface locally.

For data scientists

- The **DuckDB layer** (`warehouse/`) is built specifically so analysts run heavy queries without touching the OLTP path. `register_views` attaches Postgres via `postgres_scanner` and exposes filter-pushed-down views.
- **SQL is versioned**: `warehouse/queries/correlation_matrix.sql` and `volume_by_session.sql` are checked in and reusable across notebooks. Queries don't disappear into ad-hoc Jupyter cells.
- The `notebooks/README.md` documents how to spin up Jupyter against the live stack and the suggested layout (`01_correlation_analysis`, `02_volume_by_session`, `03_news_sentiment_alpha`, `04_anomaly_detection`).

For AI engineers

- The **news pipeline** already includes a `SearchVector` column (Postgres full-text), a `sentiment` field, and a `sentiment_score` float. Adding semantic search (pgvector embeddings) is a column-add, not a model change.
- Connector contracts return **typed dataclasses** — easy to slot a model-based normaliser between fetch and persist.
- **OpenTelemetry hooks** are wired so any downstream LLM call (e.g. classification, summarisation) inherits trace context.

For operators (on-call)

- **`docs/RUNBOOK.md`** — alert-by-alert playbook (`APIHighErrorRate`, `PipelineFreshnessStale`, `DLQGrowing`) with immediate actions, common ops (pause schedule, backfill, refresh matview, rotate API key), and recovery procedures.
- **`scripts/dlq_inspect.py`** — list / requeue / purge dead-lettered tasks from the CLI without touching the admin.

```
docs/CONTRIBUTING.md · docs/RUNBOOK.md · .github/CODEOWNERS · .github/PULL_REQUEST_TEMPLATE.md ·
warehouse/queries/ · scripts/
```

7

STRATEGY · ARCHITECTURE

Architecture strategy with the CTO

CAPTURED IN 7 ADRS

> *"Collaborate directly with the CTO to define data strategy and technical architecture as the platform scales."*

Honest framing

There is no real CTO involved here, but every consequential decision is captured in a written **Architecture Decision Record** — exactly what a CTO review session produces. ADRs make the strategy auditable, comparable across alternatives, and explicit about trade-offs.

What every ADR contains

1. **Context** — why a decision is needed now
2. **Decision** — what was chosen, briefly
3. **Rationale** — why this and not the alternatives
4. **Trade-offs** — what was given up; honest about limitations
5. **Alternatives considered** — and why each was ruled out

The seven decisions on file

ADR	TOPIC	DECISION
0001	Backend framework	Django over FastAPI standalone; accept sync-ORM trade-off, mitigate with Celery
0002	Task queue	Celery over RQ / arq for routing + retry semantics
0003	Orchestration	Dagster over Airflow / Prefect for asset model and lineage
0004	Data layout	Medallion bronze / silver / gold with bronze immutability for replays
0005	Time-series partitioning	Postgres declarative partitioning over TimescaleDB / Citus — no extension dependency, sufficient for projected scale
0006	Analytics layer	DuckDB embedded over a separate warehouse — isolation from OLTP without a CDC pipeline
0007	Logging	Structured JSON via custom formatter + ContextVar request-id propagation; OpenTelemetry traces complementary

Each ADR ends with a "*What's missing for production*" note where applicable — explicitly listing the gap between the demo and a hardened production deployment (e.g. real OIDC auth, automated backups, multi-tenancy, alerting policies).

```
docs/adr/0001-django-as-backend.md · 0002-celery-vs-rq.md · 0003-dagster-orchestration.md ·
0004-medallion-architecture.md · 0005-postgres-partitioning.md · 0006-duckdb-analytical-layer.md
· 0007-structured-logging.md
```

8 · Operations manual

Everything you need to run, test and deploy the platform.

Run locally (one command)

```
git clone https://github.com/henriquezbh5-cpu/marketintel.git
cd marketintel
cp .env.example .env
docker compose up -d
docker compose exec web python manage.py migrate
docker compose exec web python manage.py apply_post_migrate
docker compose exec web python manage.py loaddata seeds/sources.json seeds/instruments.json
docker compose exec web python manage.py shell -c \
  "from pipelines.tasks.yahoo import ingest_yahoo_spot_for_active; ingest_yahoo_spot_for_active.apply().ge
open http://localhost:8000/
```

The `scripts/bootstrap.sh` script wraps the above into a single invocation.

Run the test suite

```
# Full suite
make test

# Unit only (no DB / Redis required)
make test-unit

# Integration only (needs the running stack)
make test-integration
```

27 tests, 1 skipped (the storage test that requires the optional `moto` dependency for S3 mocking).

Common operations

NEED TO DO	HOW
Issue an API key	`python scripts/issue_api_key.py <username> --scope read`
Inspect the dead-letter queue	`python scripts/dlq_inspect.py list`
Requeue a failed task	`python scripts/dlq_inspect.py requeue <task_id>`
Backfill a date range	`./scripts/backfill.sh 2026-04-01 2026-04-15`
Pull fresh quotes manually	`make ingest`
Take a Postgres backup	`./scripts/backup.sh 30` (30-day retention)
Smoke-test a running deploy	`./scripts/healthcheck.sh https://your-host`

Deploy to production

Three documented paths in `docs/DEPLOY.md`:

1. **Render** (recommended for free permanent demo). Connect repo, click "Apply Blueprint" — the `render.yaml` provisions everything. Seed data via the Shell tab. Free tier sleeps after 15 min idle.
2. **Railway** for a $5-credit, faster-build alternative. Pick repo, add Postgres + Redis plugins, add a worker service with the celery start command. `railway.toml` is committed.
3. **Kubernetes** for production. `kubectl apply -k infra/k8s/overlays/production` applies Deployments + Services + HPA + PDB + NetworkPolicy + Ingress + migrations Job.

What to monitor in production

METRIC	THRESHOLD	PAGE
API 5xx rate	> 5% for 10 min	`APIHighErrorRate`
API p95 latency	> 1s for 10 min	`APILatencyP95High`
Spot freshness	> 5 min since last ingest	`PipelineFreshnessStale`
DLQ growth	> 10 inserts in 15 min	`DLQGrowing`

10 · Live demo · what each page shows

PATH	WHAT IT SHOWS	WHAT TO LOOK FOR
`/`	Overview	Featured chart updates every 5s. Top movers tabs work. Watchlist sparklines render after first data load. Status cards bottom-right reflect live freshness.
`/markets/`	All instruments	Search filters as-you-type. Sort buttons toggle direction on second click. Each row links to detail.
`/markets/AAPL/`	Symbol detail	Resolution selector (1m–1d) re-queries the partitioned fact. Window selector (6h–7d) adjusts the time range. Toggle between line and candlestick chart types.
`/news/`	News feed	Empty until `CRYPTOPANIC_API_KEY` is set — the page handles the empty state explicitly.
`/system/`	Pipeline status	Source connectivity badges (LIVE/STALE). DLQ counter. Partition table with row distribution. Architecture diagram (bronze/silver/gold).
`/coverage/`	Job-spec mapping	Bullet-by-bullet, what was delivered, where to see it, source files.
`/api/docs/`	Swagger UI	OpenAPI 3.1 schema, all endpoints, auth flow. Click *Authorize*, paste API key, try endpoints.
`/health/ready/`	Readiness probe	JSON with per-dependency status. 503 if degraded.

11 · Source-code map

```
marketintel/
├── apps/
│   ├── api/             REST API (DRF viewsets, serializers, throttling, auth)
│   ├── core/            Shared models (DLQ), middleware, health probes, metrics
│   ├── dashboard/       Server-rendered web UI (templates + JSON endpoints)
│   ├── instruments/     Source + Instrument (SCD Type 2)
│   ├── prices/          Partitioned candle fact + spot quote table
│   └── news/            News articles with sentiment + full-text index
├── pipelines/
│   ├── connectors/      HTTP clients for third-party APIs
│   │   ├── base.py            Token bucket + circuit breaker + retries + metrics
│   │   ├── yahoo_finance.py   Active source · equities + indices + ETFs
│   │   ├── binance.py         Alternate · crypto OHLCV
│   │   ├── coingecko.py       Alternate · crypto spot
│   │   └── cryptopanic.py     Alternate · news with sentiment
│   ├── tasks/           Celery tasks (idempotent, retry-aware, DLQ-backed)
│   │   ├── common.py          IdempotentTask base + on_failure hook
│   │   ├── yahoo.py           Yahoo ingest tasks
│   │   ├── ingest.py          Crypto ingest tasks
│   │   ├── transform.py       Gold matview refresh
│   │   └── maintenance.py     Future-partition creation
│   └── orchestration/   Dagster assets, jobs, schedules, sensors
├── warehouse/           DuckDB analytics layer + sample SQL queries
├── config/              Django settings (base/dev/prod/test) + celery + observability
├── infra/
│   ├── postgres/        init SQL + post-migrate gold marts
│   ├── k8s/             Kustomize base + staging/production overlays
│   ├── grafana/         Pre-built dashboards + datasource provisioning
│   └── prometheus/      Scrape config + alert rules
├── docs/
│   ├── adr/             7 Architecture Decision Records
│   ├── data-model.md    Schema reference
│   ├── pipelines.md     Cadence, idempotency contract, failure handling
│   ├── DEPLOY.md        Tunnel · Render · Railway · K8s
│   ├── RUNBOOK.md       On-call playbook
│   ├── CONTRIBUTING.md  Setup + workflow
│   └── MANUAL.html      This document
├── tests/               27 tests · unit + integration + HTTP mocks
├── scripts/             ops scripts (bootstrap, backfill, dlq, expose, etc.)
├── seeds/               Initial Source + Instrument data
├── examples/            Python client, cURL, Postman collection
├── .github/             Workflows (CI, deploy, codeql) + templates
├── .devcontainer/       VS Code devcontainer
├── Dockerfile           Multi-stage (dev + prod targets)
├── docker-compose.yml   Local 7-container stack
├── render.yaml          Render deploy blueprint
├── railway.toml         Railway deploy hint
├── pyproject.toml       Python dependencies + tooling
└── Makefile             Common dev tasks
```

Closing note

Everything in this manual is verifiable in code, in the live demo at `github.com/henriquezbh5-cpu/marketintel`, or both. Where a requirement could not be fully demonstrated solo (collaboration, working with a real CTO), the document calls that out explicitly rather than overclaiming.

— end of manual —